CLASE 5 · TRACK DE IA

RAG vectorial

Búsqueda por significado con FAISS y LangChain

De los vectores a una respuesta con contexto

Indexar → Recuperar → Generar

AGENDA

Objetivos de la clase

1. **Ver el pipeline completo de un RAG vectorial**
   Las dos fases, indexación y consulta, y qué pasa en cada una.

2. **Entender qué es una base de datos vectorial**
   Dónde se guardan los chunks con sus vectores y cómo se buscan.

3. **Recuperar por similitud con un retriever**
   Cómo se eligen los chunks más cercanos a la pregunta.

4. **Generar la respuesta con RetrievalQA**
   La cadena que une búsqueda, prompt y modelo en un solo paso.

DE DÓNDE VENIMOS

Lo que tenemos y lo que falta

LO QUE YA TENEMOS

Clase 4

- Documentos divididos en chunks
- Cada chunk convertido en vector
- Cercanía = significado parecido

HOY AGREGAMOS

Clase 5

- Guardar textos y vectores juntos
- Buscar los más cercanos a la pregunta
- Entregárselos al LLM para responder

Ninguna pieza es nueva: hoy las conectamos en un sistema que responde.

01

El pipeline completo

Dos fases que ocurren en momentos distintos

FLUJO

Dos fases, dos momentos

INDEXACIÓN · se hace una sola vez

1. Cargar el documento
2. Dividir en chunks
3. Generar embeddings
4. Guardar en la base vectorial

CONSULTA · se repite en cada pregunta

1. Llega la pregunta
2. Embedding de la pregunta
3. Búsqueda de similitud
4. Chunks como contexto
5. El LLM responde

La consulta busca en lo que guardó la indexación: esa base une las dos fases.

CONCEPTO

Dos modelos, dos trabajos

MODELO DE EMBEDDINGS

Convierte texto en vector

Trabaja en las dos fases: al indexar cada chunk y al recibir cada pregunta.

En el notebook: vía OpenRouter

MODELO DE CHAT (LLM)

Redacta la respuesta

Solo aparece al final de la consulta, cuando ya tiene la pregunta y los chunks.

En el notebook: vía Groq

Regla: la pregunta se convierte con el mismo modelo de embeddings que los chunks. Vectores de modelos distintos no se pueden comparar.

02

Guardar y buscar

Base de datos vectorial y búsqueda por similitud

DEFINICIÓN

¿Qué es una base vectorial?

Un almacén que guarda cada chunk junto a su vector y permite encontrar rápido los vectores más cercanos a uno dado.

TEXTO DEL CHUNK	VECTOR
“La IA nace como campo en 1956…”	`[0.12, -0.87, 0.33, …]`
“El primer invierno llegó en los 70…”	`[-0.45, 0.08, 0.91, …]`
“Hoy vivimos la era de los LLM…”	`[0.67, 0.21, -0.14, …]`

valores ilustrativos: los vectores reales tienen cientos de números

FAISS

Facebook AI Similarity Search

Librería de Meta optimizada para buscar similitud entre millones de vectores.

En el notebook vive en memoria: si reinicias el kernel, se vuelve a crear.

INDEXACIÓN

Lo que pasa al crear la base

En el notebook es una sola línea, pero por dentro hace tres cosas:

1 Recibe chunks

La lista de fragmentos que armamos con el splitter.

2 Los vectoriza

Genera un embedding por cada fragmento con el modelo de embeddings.

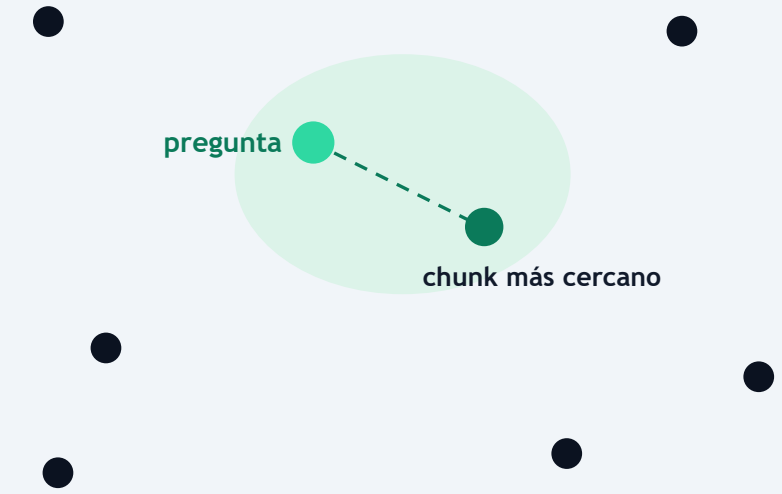
3 Arma el índice

Guarda cada vector junto a su texto, listo para buscar.

Más chunks = más embeddings: por eso se indexa una vez, no en cada pregunta.

INTUICIÓN

Buscar por cercanía



representación en 2D, simplificada

1. La pregunta se convierte en vector
2. Se mide su distancia a cada chunk
3. Se ordenan de más a menos cercano
4. Se devuelven los k más cercanos

Coseno mide el ángulo; euclidiana, la distancia. FAISS usa euclidiana por defecto.

PARÁMETRO

El retriever y el valor de k

El retriever recibe la pregunta y devuelve los k chunks más cercanos desde la base vectorial. El valor de k lo defines tú.

1 k chico

Contexto preciso y prompt corto. Si la respuesta está repartida en varios chunks, se pierde una parte.

2 k grande

Más cobertura, pero entra ruido y el prompt crece: más tokens y más costo.

3 k = 1

Es lo que usa el notebook. Alcanza porque cada pregunta se responde con un solo fragmento.

03

Generar la respuesta

RetrievalQA y la estrategia stuff

ORQUESTACIÓN

RetrievalQA une todo

LE ENTREGAS

LLM
El modelo que redacta la respuesta

Retriever
El buscador sobre la base FAISS

Tipo de cadena
Cómo se arma el prompt (stuff)

LA CADENA HACE

1. Toma la pregunta
2. Busca los chunks relevantes
3. Arma el prompt: pregunta + chunks
4. Lo envía al LLM y responde

Una sola llamada a la cadena ejecuta los cuatro pasos.

ESTRATEGIA

Stuff: rellenar el prompt

EL PROMPT QUE LLEGA AL LLM

Instrucción
Usa el contexto para responder. Si no sabes, di que no sabes.

Contexto
Los chunks recuperados, pegados tal cual.

Pregunta
Lo que escribió el usuario.

La instrucción la pone LangChain por defecto.

1 **Qué hace**

Mete todos los chunks recuperados en un solo prompt, sin resumirlos.

2 **A favor**

Es la estrategia más simple: una sola llamada al LLM.

3 **Límite**

Funciona mientras los chunks quepan en la ventana de contexto del modelo.

LÍMITES

¿Y si la respuesta no está?

1 No filtra

El retriever entrega los k más cercanos aunque ninguno tenga que ver con la pregunta.

2 Instrucción

La plantilla le pide al modelo decir que no sabe si el contexto no alcanza.

3 Sin garantía

RAG reduce las alucinaciones, no las elimina: el modelo igual puede completar con lo que sabe.

RAG no hace que el modelo sepa más: le da mejor contexto.

AL NOTEBOOK

Ahora lo armamos en código

`4-vector-rag.ipynb`

FAISS · Retriever · RetrievalQA